Using GenAI Agents to Reconstruct Single Function Library

The University of Osaka
Warren Gerald Polandra – M1, October Enrollment
Raula Gaikovina Kula, Yoshiki Higo

大阪大学
THE UNIVERSITY OF OSAKA

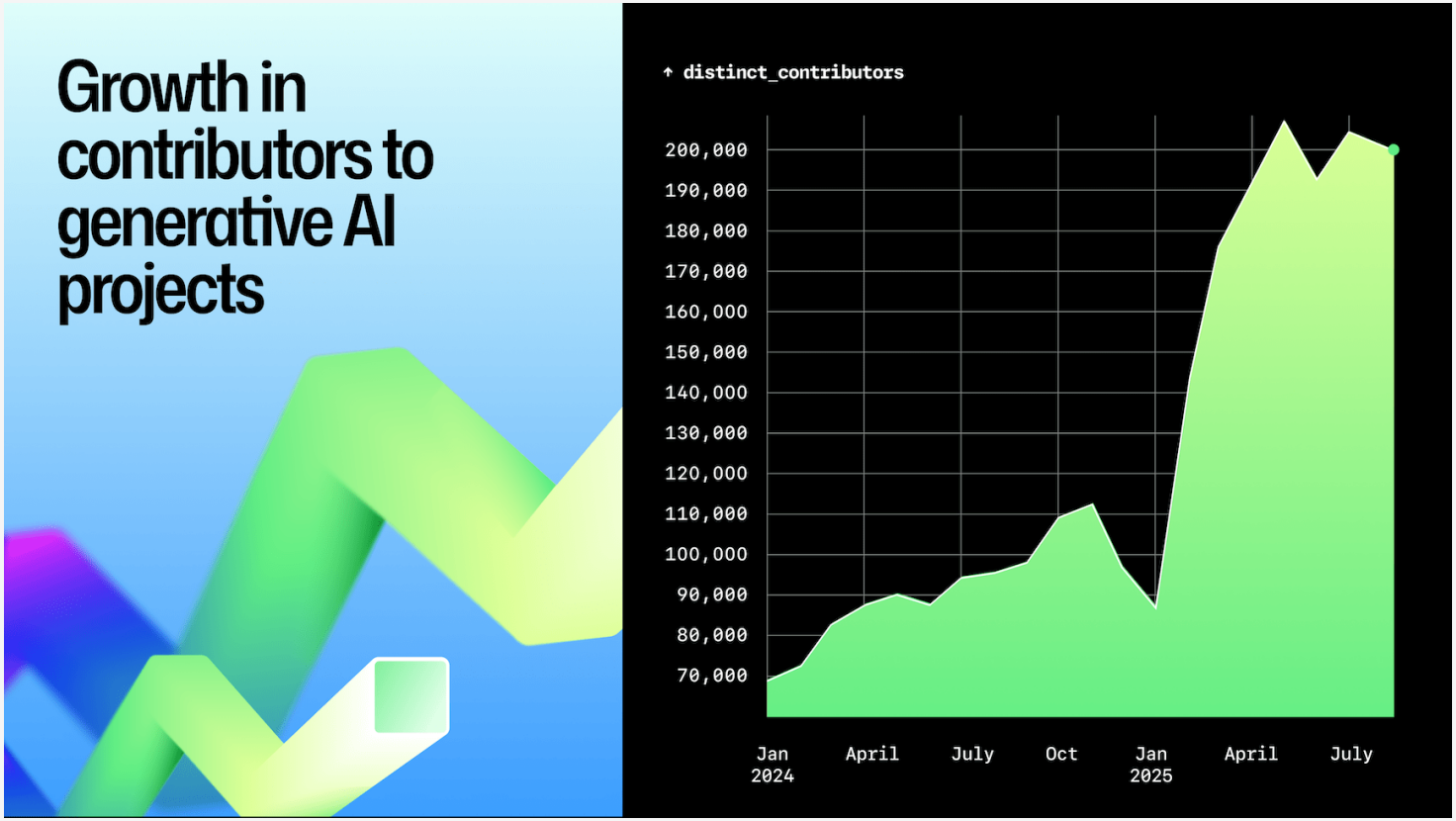


Presentation Overview

1. Background
   a. GenAI in Software Development
   b. Trivial & Micro Packages
2. Key Ideas
3. Experiments
   a. Datasets
   b. Method
4. Initial Results
5. Future Works



Background – Gen AI Projects



Source: GitHub Octoverse[1]

Background – GitHub Copilot



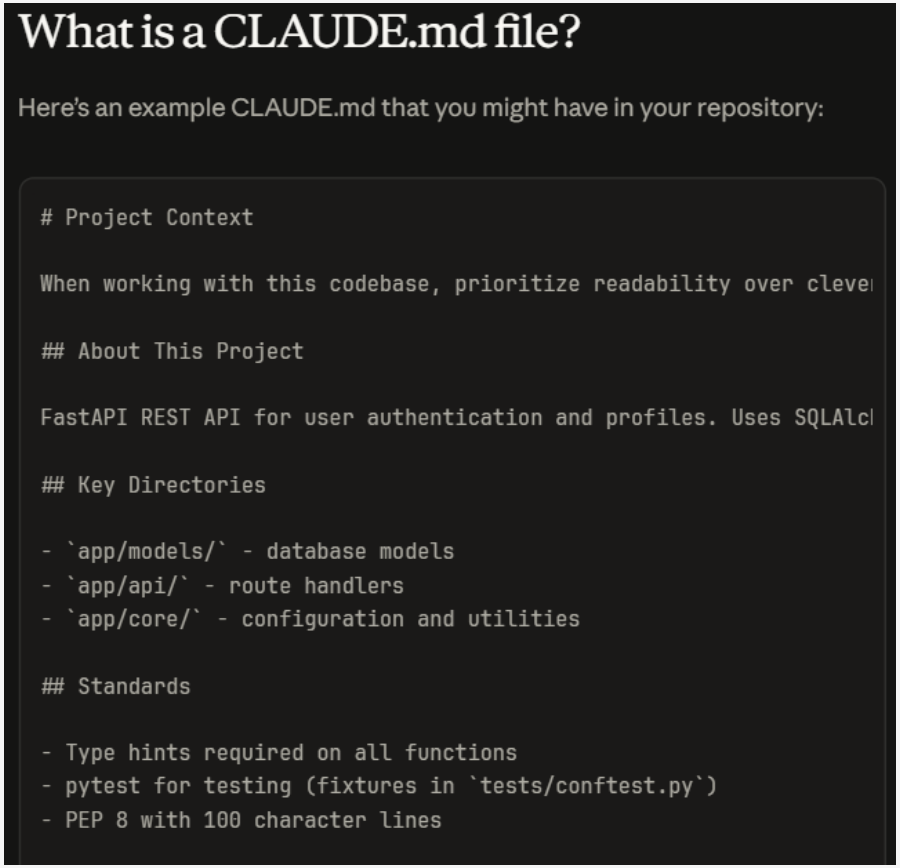
According to an interview conducted by GitHub,

72.6% developers found that Copilot **improved their code review effectiveness**[2]

Background – Agents Specifications



AGENTS.md file[3]



CLAUDE.md file[4]

Background – Specification Structures

Table 1: Categories, descriptions, and their prevalence.

Category	Label	Description	%
General	`System Overview`	Provides a general overview or describes the key features of the system.	48.2
	`AI Integration`	Contains instructions or notes specifically for integrating with or interacting with the agentic coding tools.	15.4
	`Doc.&Refs`	Lists supplementary documents, links, or references for additional context.	13.8
Impl.	`Architecture`	Describes the high-level structure, design principles, or key components of the system's architecture.	64.8
	`Impl. Details`	Provides specific details for implementing code or system components, including coding style guidelines.	71.9
Build	`Build and Run`	Outlines the process for compiling source code and running the application, often including key commands.	77.1
	`Testing`	Details the procedures and commands for executing automated tests.	60.5
	`Conf.&Env.`	Instructions for configuring the system and setting up the development or production environment.	26.5
	`DevOps`	Covers procedures for software deployment, release, and operations, such as CI/CD pipelines.	9.5
Management	`Development Process`	Defines the development workflow, including guidelines for version control systems like Git.	37.2
	`Project Management`	Information related to the planning, organization, and management of the project.	11.1
Quality	`Maintainance`	Guidelines for system maintenance, including strategies for improving readability, detecting and resolving bugs.	19.8
	`Performance`	Focuses on system performance, quality assurance, and potential optimizations.	12.7
	`Security`	Addresses security considerations, vulnerabilities, or best practices for the system.	8.7
	`UI/UX`	Contains guidelines or details concerning the user interface (UI) and user experience (UX).	8.3

W. Chatlatanagulchai et al., 2025[5]

Background – Prompt Structure

You are an autonomous agent.
You are working towards the objective on a step-by-step basis

OBJECTIVE: {objective} (e.g. "Find a recipe for chocolate chip cookies")

Previous steps:
{context}

Your task is to respond with the next action.

Supported commands are:
command | argument
---
memorize_thoughts | internal debate, refinement, planning
execute_python | python code (multiline)

The mandatory action format is:
<r>[YOUR_REASONING]</r><c>[COMMAND]</c>

Use your existing knowledge rather then web search when possible.
Use memorize_thoughts to organize your thoughts (to be stored in memory)
DO NOT CHAIN MULTIPLE COMMANDS.
DO NOT REPEAT PREVIOUSLY EXECUTED COMMANDS.

Example actions:

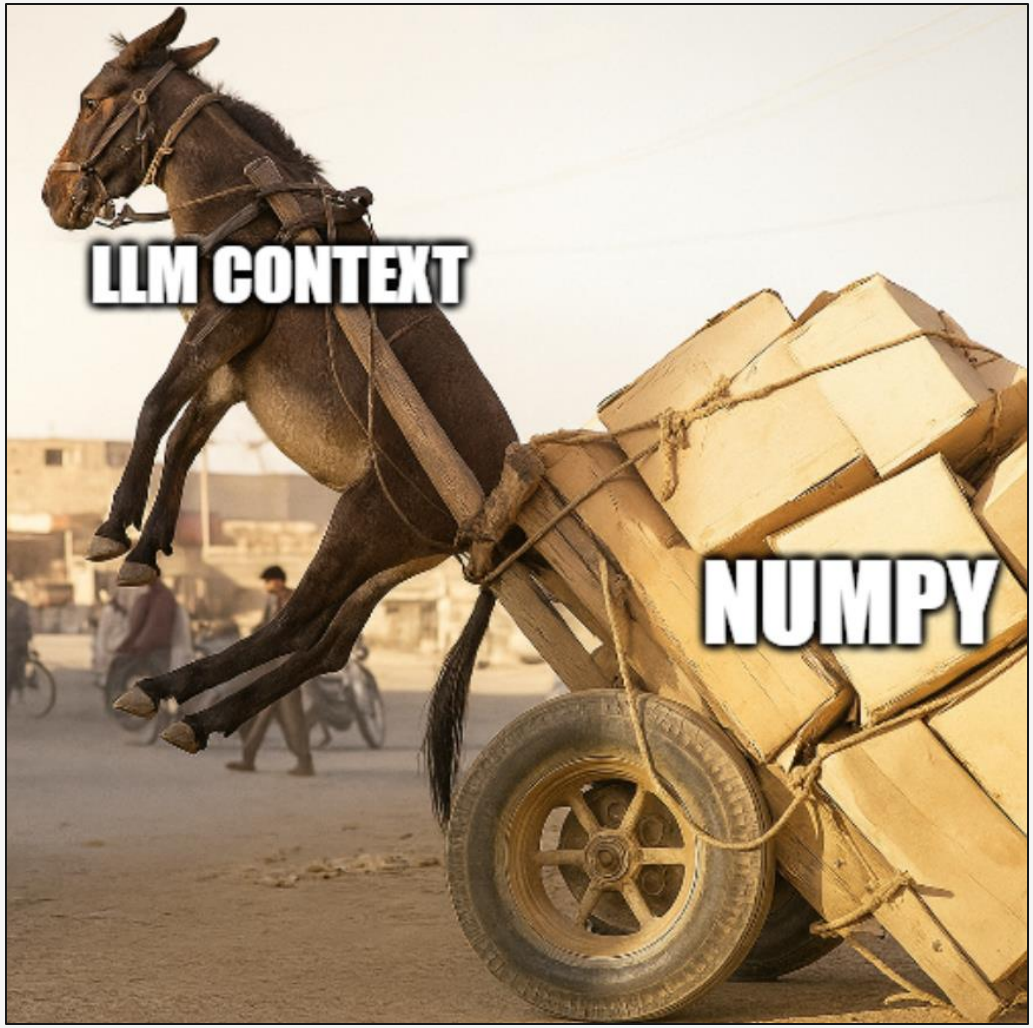
<r>Search for websites with chocolate chip cookies recipe.</r><c>web_search</c>

<r>Scrape information about chocolate chip cookies from the given URL.</r><c>web_scrape</c>



M. Tafreshipour et al., 2025[6]

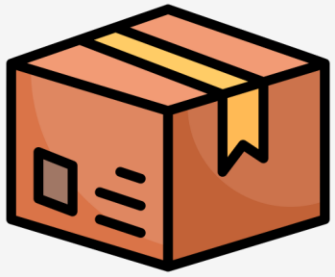
Background – Library Documentation



Background – Trivial & Micro-packages

```
1  'use strict';
2  module.exports = function (n) {
3      return toString.call(n) === '[object Number]' && n > 0;
4  };
5
```

Example of a Trivial Package[7]



Micro Package[8]

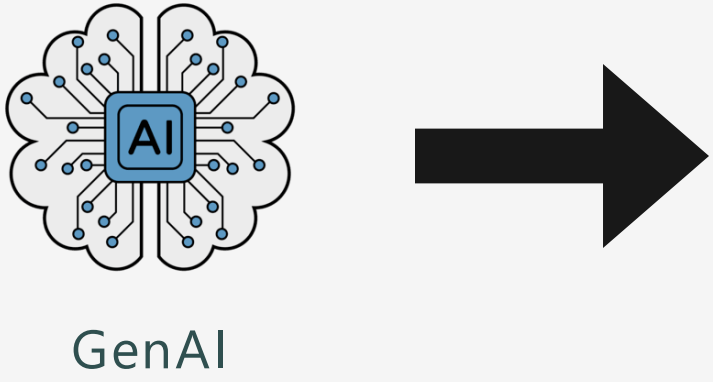
```
1  'use strict';
2  module.exports = function (x) {
3    return x === 0 && 1 / x === -Infinity;
4  };
```

Perform a Trivial Task

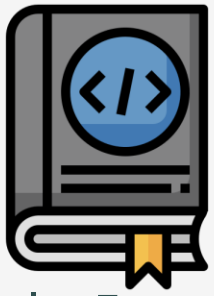
```
1  'use strict';
2  module.exports = require('os-homedir')();
```

Façade to load foreign module dependencies

Key Ideas



GenAI



Single Function Library



Specification



Reconstruct

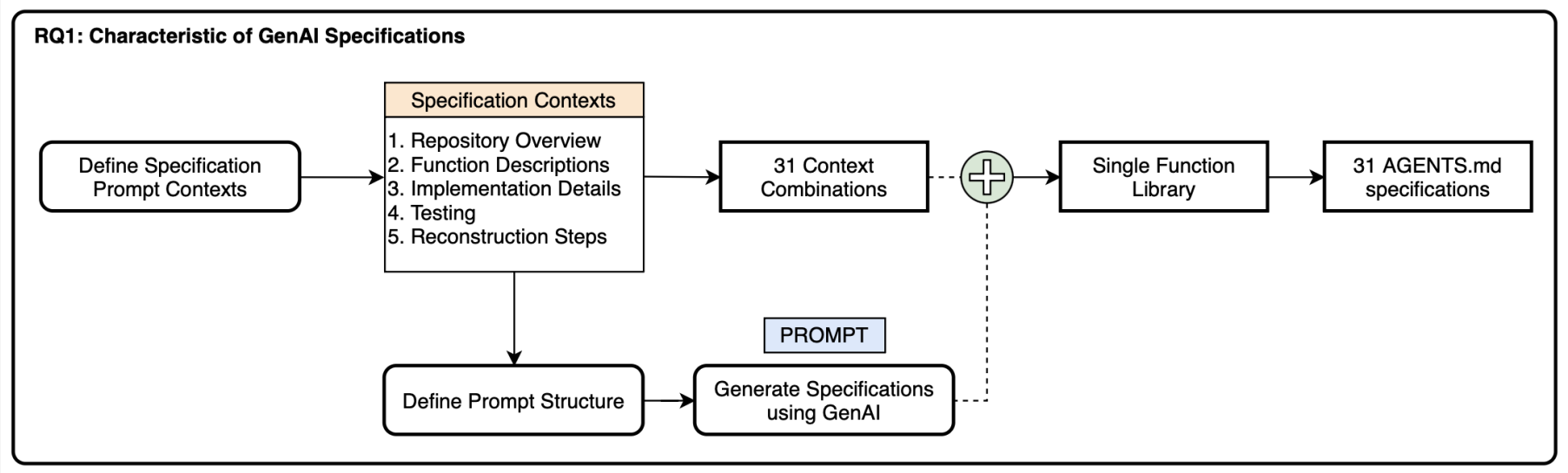


Single Function Library

Research Questions

1. What is the characteristic of specifications created by GenAI?
   a. Which context combination created the most specification?
   b. Which context combination created the least specification?

Methodology



IDE, AI-Agent & LLM

IDE – Visual Studio Code



Agent – GitHub Copilot



LLM – GPT 4.1



PyPI Single Function Library



A simple python library that generates concise, unambiguous, URL-safe UUIDs[9]

271 Lines of Code, 4 files, 7 public APIs (excluding testcases)

https://pypi.org/project/shortuuid/

ShortUUID – Examples

```
>>> import shortuuid
```

```
>>> shortuuid.uuid()
'vytxeTZskVKR7C7WgdSP3d'
```

```
>>> shortuuid.uuid(name="example.com")
'exu3DTbj2ncsn9tLdLWspw'

>>> shortuuid.uuid(name="<http://example.com>")
'shortuuid.uuid(name="<http://example.com>")'
```

```
>>> shortuuid.ShortUUID().random(length=22)
'RaF56o2r58hTKT7AYS9doj'
```

Data Collection – Prompt & Contexts

You are an autonomous software-documentation agent…

You are tasked with generating documentation for a code repository…

{Contexts}

- DO NOT WRITE THE CONTENT/LOGICS OF THE FUNCTION FROM THE REPOSITORY
- DO NOT MENTION THE NAME OF THE REPOSITORY OR THE RELATED LINKS

Role **Directive** **Consideration**

Additional Information/Context

1. Repository Overview — General
2. Function Descriptions — Impl.
3. Implementation Details — Impl.
4. Testing — Build.
5. Reconstruction Steps — Build.

$$Total\ Combinations = \sum_{i=1}^{5} C(5, i) = \mathbf{31}$$

Initial Results – Specification Stats

Table 1. Character Count Statistics

Order	Character Count	Contexts Combination
1	14,222	2-3.md
2	12,885	2-4.md
3	12,533	2-3-5.md
29	2,807	1-5.md
30	2,751	3-4.md
31	2,697	1-4.md
Average	6,818.84	

Table 2. Lines of Docs Statistics

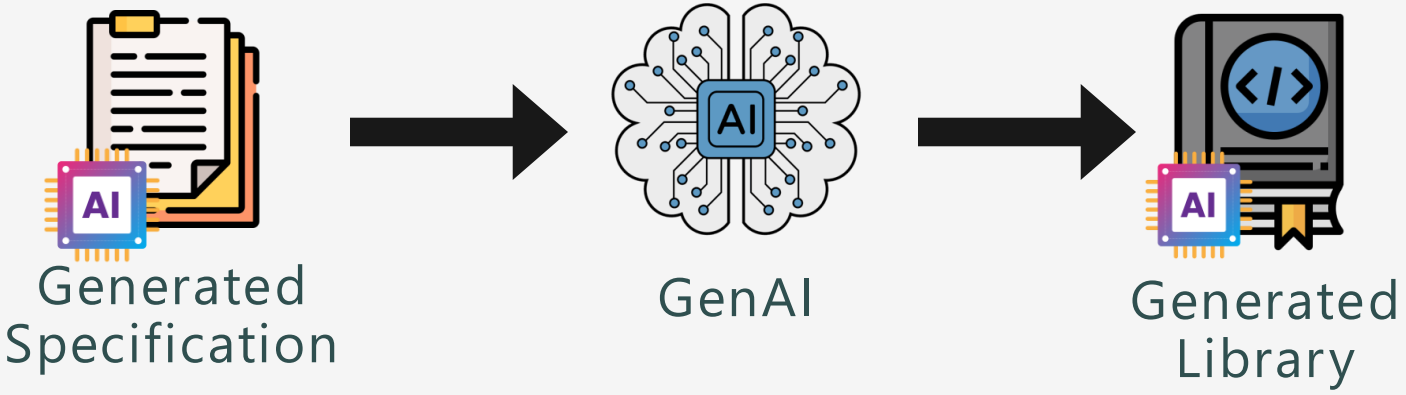
Order	Lines of Docs	Context Combination
1	567	2-4.md
2	478	2-3.md
3	385	2-4-5.md
29	81	1-3-4-5.md
30	78	1-4.md
31	64	3-4.md
Average	213.97	

Context Labels:

1. Repository Overview
2. Function Descriptions
3. Implementation Details
4. Testing
5. Reconstruction Steps

Future Works

1. Reconstruct Single Function Library using generated specification
2. Comparison between generated Single Function Library with the original one



References

[1] https://octoverse.github.com

[2] https://github.blog/ai-and-ml/generative-ai/code-review-in-the-age-of-ai-why-developers-will-always-own-the-merge-button

[3] https://agents.md

[4] https://www.claude.com/blog/using-claude-md-files

[5] Chatlatanagulchai, Worawalan, et al. “On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code.” (2025)

[6] Tafreshipour, Mahar, et. al. “Prompting in the Wild: An Empirical Study of Prompt Evolution in Software Repositories.” (2025)

[7] Abdalkareem, Rabe, et. al. “Why Do Developers Use Trivial Packages? An Empirical Case Study on npm.” (2017)

[8] Kula, Raula Gaikovina, et. al. “On the Impact of Micro-Packages: An Empirical Study of the npm JavaScript Ecosystem.” (2017)

[9] https://pypi.org/project/shortuuid/

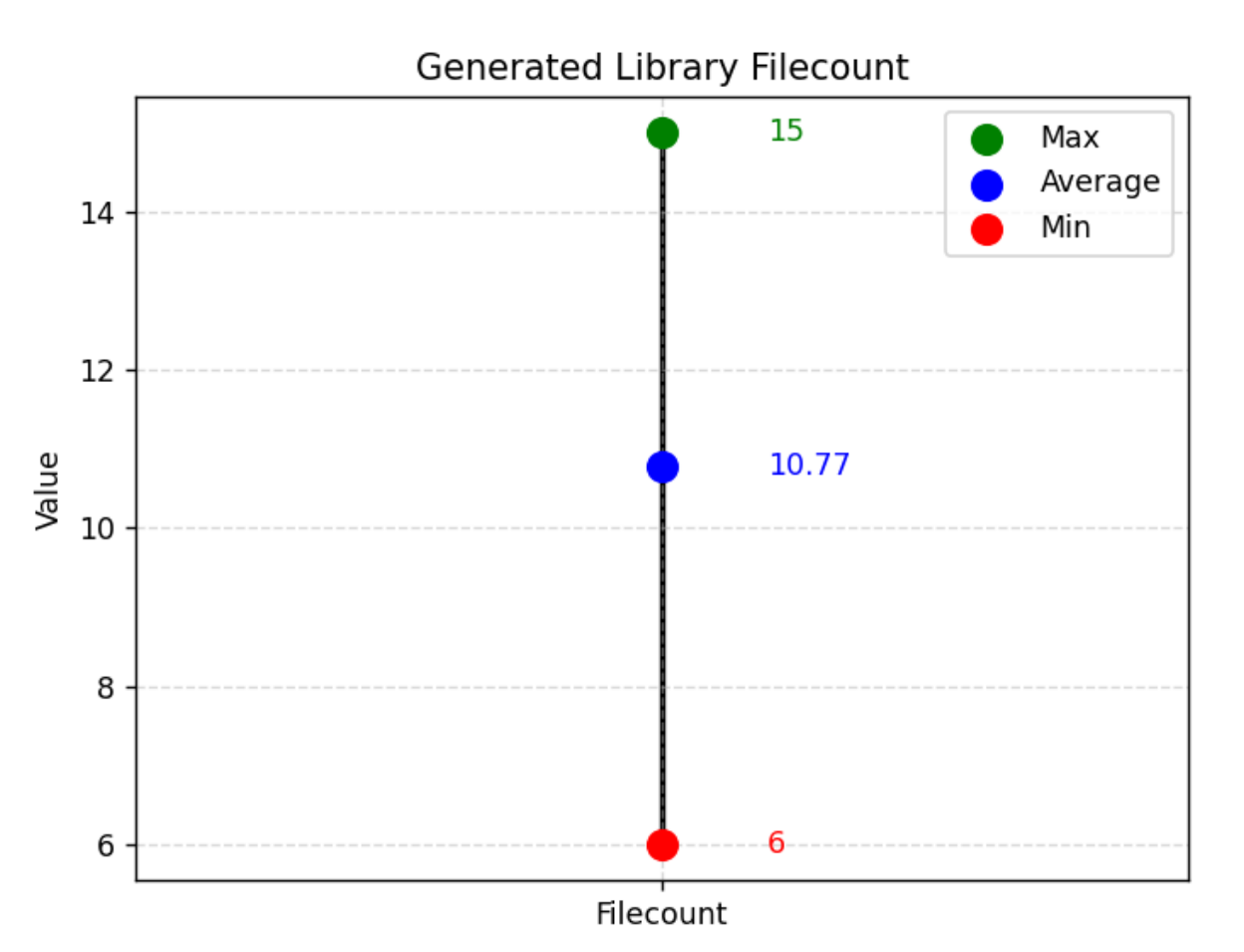
Using GenAI Agents to Reconstruct Single Function Library

The University of Osaka
Warren Gerald Polandra – M1, October Enrollment
Raula Gaikovina Kula, Yoshiki Higo

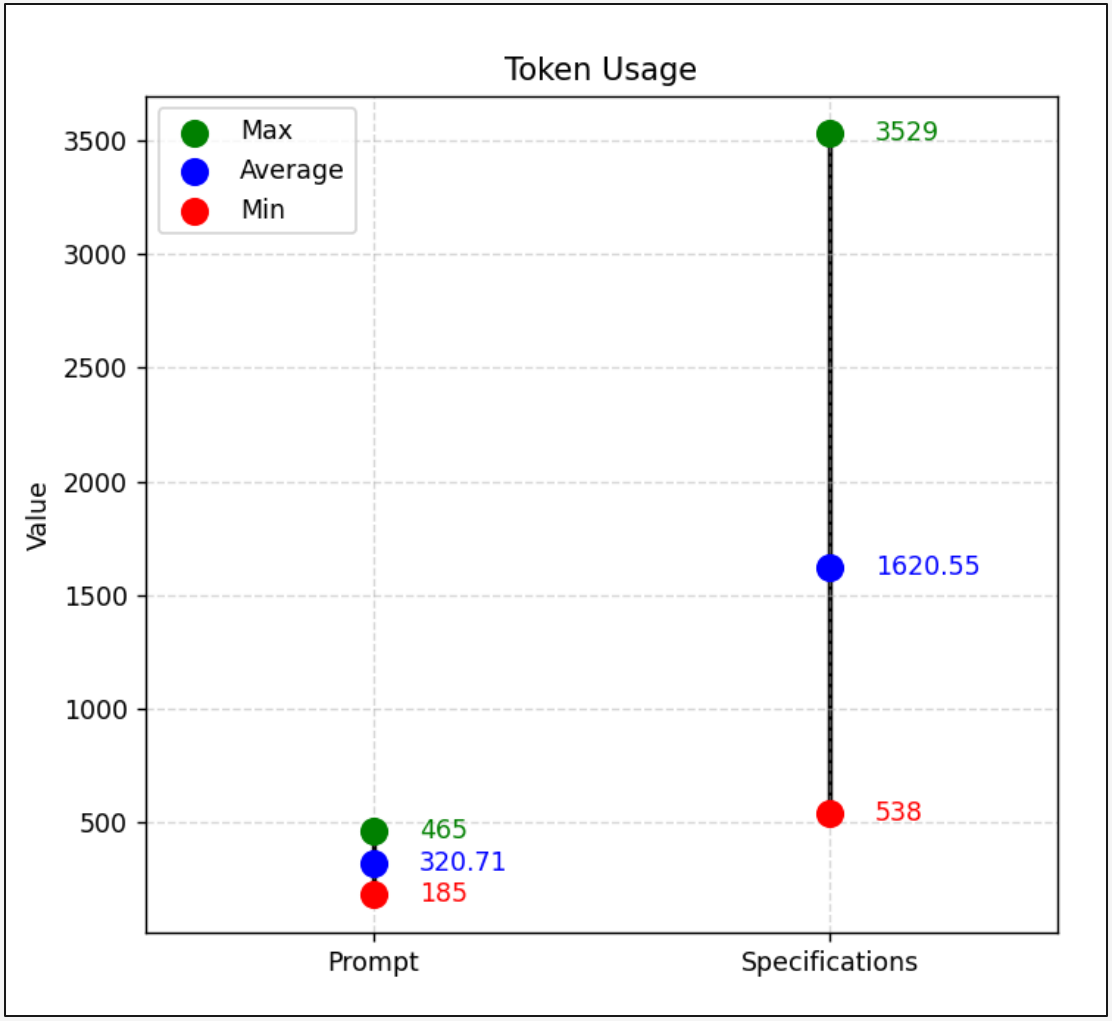
大阪大学
THE UNIVERSITY OF OSAKA



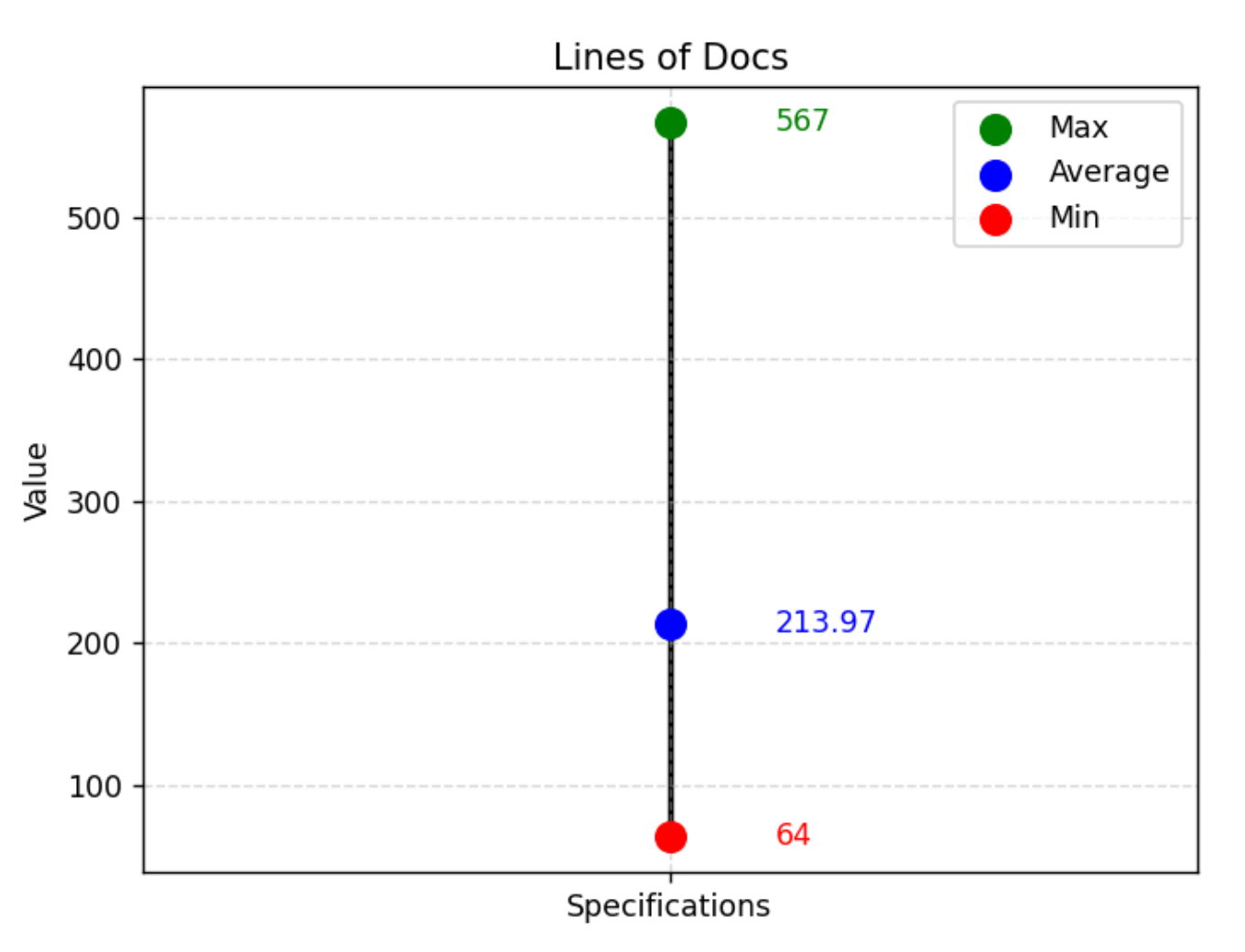
Supplementary Materials



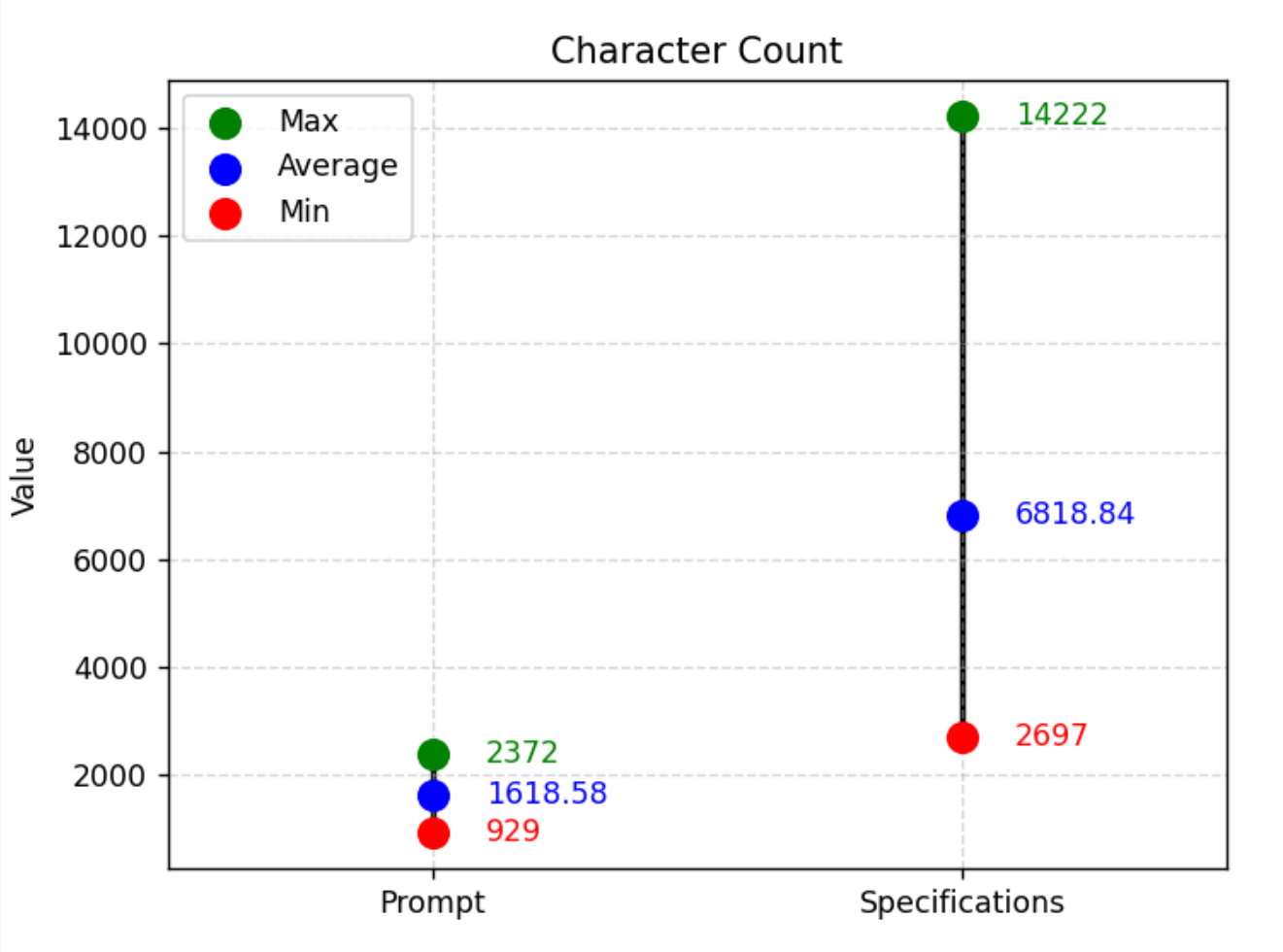
Supplementary Materials – Token Usage



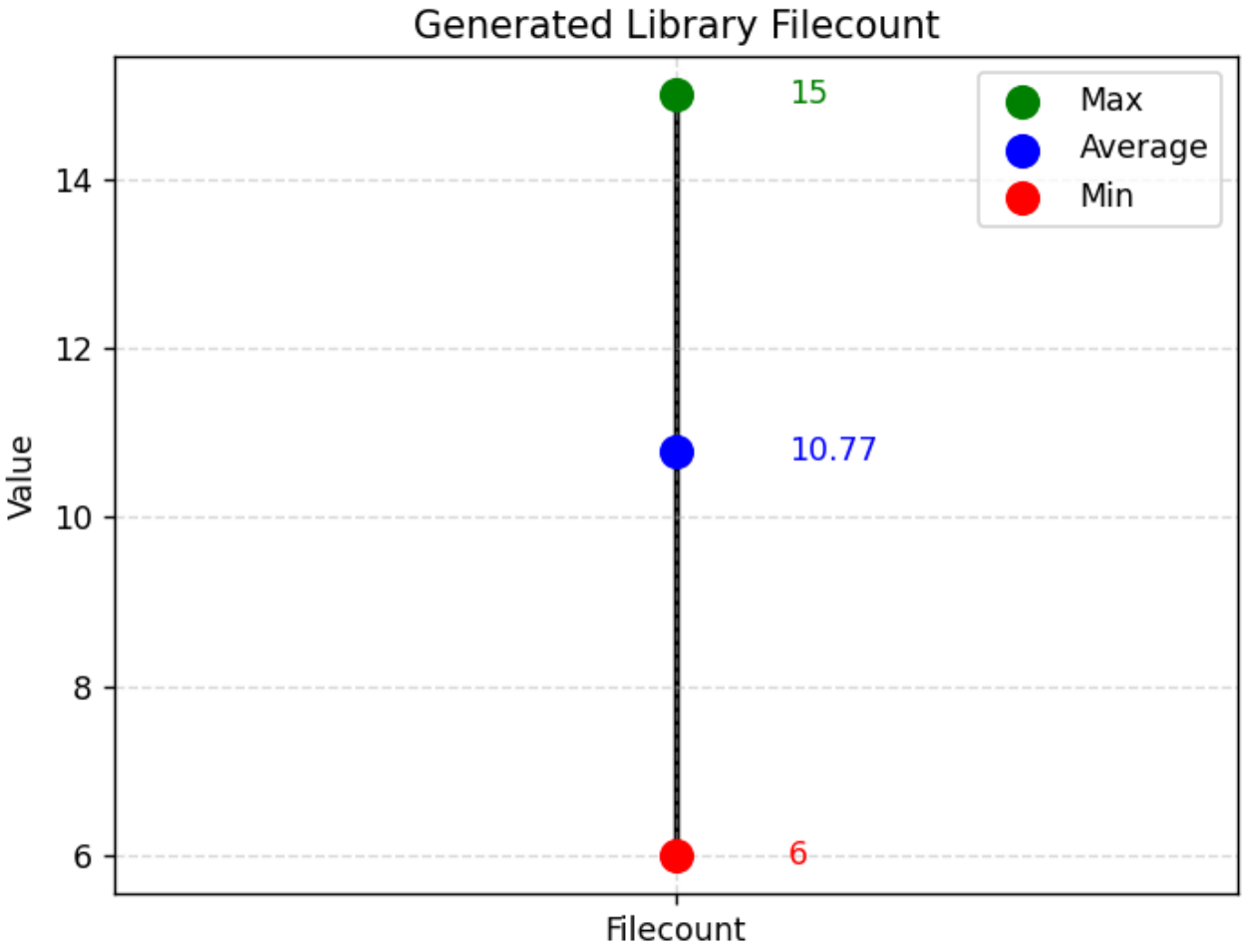
Supplementary Materials – Lines of Docs



Supplementary Materials – Character Count



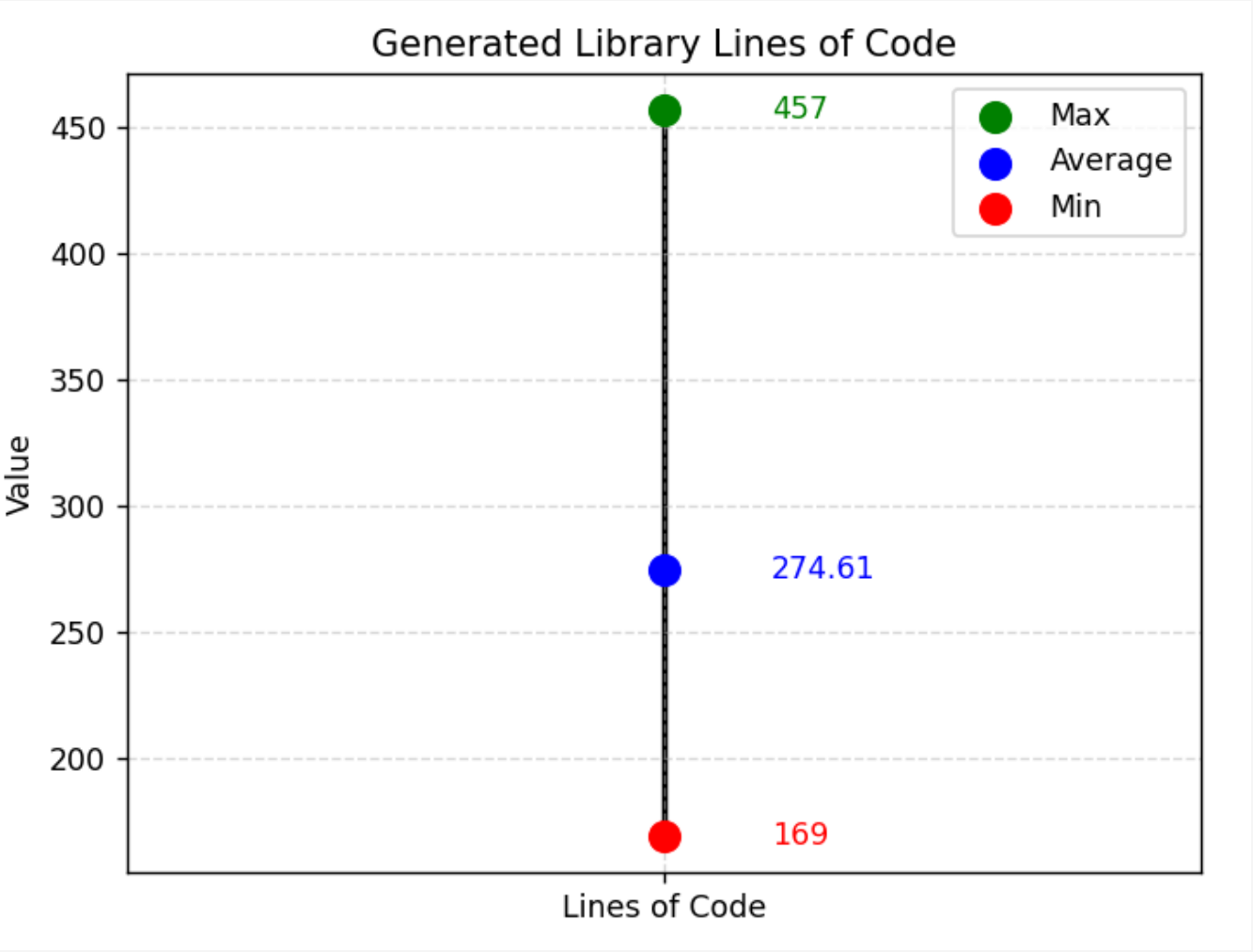
Generated Library File count



Original File count:

57

Generated Library Lines of Code



Original Library LoC:
492